TP Projet 0 : Fondations communes

HESTIM | Data Governance | Projet "Enterprise Multi-Agent AI Governance Platform"
À faire : les 3 membres ensemble | Durée indicative : 1 séance

---

Objectifs pédagogiques

À l'issue de ce TP, vous serez capables de :

- Expliquer en une phrase le but du projet et le rôle de chaque membre
- Faire la différence entre Data Management et Data Governance
- Citer les 3 piliers de la gouvernance et les rôles Owner, Steward, Custodian
- Expliquer les 4 types de métadonnées et la différence catalogue / dictionnaire
- Expliquer ce qu'est une Master Data, un MDM et un Golden Record
- Lancer l'environnement commun (PostgreSQL + OpenMetadata) sur votre machine
- Travailler à trois sur le même dépôt GitHub sans vous marcher dessus

Ce que l'on fait dans cette première partie du projet : on s'arrête au MDM. La Governance Engine (FastAPI), les agents IA, les tests de sécurité et le dashboard viendront dans une deuxième partie.

---

1. Le projet en une image



Principe fondamental du sujet : un agent IA n'accède **jamais** directement aux données. Mais pour qu'un portier puisse décider, il faut d'abord savoir **ce que sont les données**, **à qui elles appartiennent** et **à quel point elles sont sensibles**. C'est exactement ce que l'on construit dans la partie 1.

---

2. Qui fait quoi

	Junior	Ivan	Mohammed
Agent	Finance Agent	Investment Agent	HR Agent
Dataset	SEC Financial Statement Data	S&P 500 Historical Market Data	IBM HR Analytics
Classification	Public	Internal	Confidential
Sensibilité	Low (Faible)	Medium (Moyenne)	High (Élevée)
PII	Non	Non	Oui
MDM	**Tiers et Structures**	**Produits**	**Ressources**
Entités maîtres	Company, plan de comptes XBRL, Currency, Country	Security (titre boursier), Exchange, Sector, Product, Investor	Employee, Department, JobRole, Office
Data Owner	Direction Finance	Direction Investissement	Direction RH
TP à suivre	TP1_Finance	TP2_Investment	TP3_HR

Les 4 types de MDM viennent du chapitre 2 du cours : **Tiers** (clients, fournisseurs, partenaires), **Structures** (organisation, plan comptable), **Produits** (ce que l'on vend ou achète), **Ressources** (personnes, équipements, sites).

> **POURQUOI TROIS DATASETS SI DIFFÉRENTS ?**
> Dans la réalité, seul le lien **Company / Ticker** relie naturellement SEC et S&P 500. Le dataset RH est isolé. C'est normal : le but est de pratiquer **la même démarche de gouvernance** sur trois niveaux de sensibilité (Public, Internal, Confidential). Les quelques tables simulées (Product, Investor, Office) restent petites et servent seulement à illustrer le MDM.

---

3. Concepts fondamentaux

> **DÉFINITION : Data Management**
> La gestion **technique** des données : stocker, charger, transformer, sauvegarder. Exemple : charger un CSV dans PostgreSQL.

> **DÉFINITION : Data Governance**
> La définition et le contrôle des **règles** sur les données : qui est responsable, qui peut y accéder, quelle qualité est exigée, comment on trace. La gouvernance **ne traite pas** la donnée, elle **décide qui a l'autorité** dessus.

3.1 Les 3 piliers de la gouvernance

Pilier	Question	Dans notre projet
Personnes	Qui est responsable ?	Data Owner, Data Steward, Data Custodian pour chaque dataset

Pilier	Question	Dans notre projet
Processus	Quelles règles ?	Politiques (ex. "tout dataset a un Owner"), standards (nommage, formats), procédures (comment on charge, comment on classe)
Outils	Avec quoi ?	PostgreSQL, OpenMetadata, puis la Governance Engine

3.2 Les rôles

Rôle	Ce qu'il fait	Dans notre projet
Data Owner	Responsable métier de la donnée. Décide des règles et des accès. **Toujours une direction métier, jamais l'IT.**	Direction Finance / Investissement / RH
Data Steward	Garant au quotidien : qualité, documentation, règles de rapprochement.	Chaque membre, pour son dataset
Data Custodian	Gère la technique : base, sauvegardes, accès techniques.	L'équipe projet (celui qui gère Docker/PostgreSQL)
Data Consumer	Utilise la donnée.	Les agents IA et les managers (partie 2)

Matrice RACI (R = réalise, A = approuve, C = consulté, I = informé) pour une activité de notre projet :

Activité	Owner (Direction)	Steward (membre)	Custodian (équipe)
Classer un dataset (Public, Internal...)	A	R	I
Charger le dataset dans PostgreSQL	I	C	R
Écrire le dictionnaire de données	C	R / A	I
Définir la règle de rapprochement MDM	A	R	C

3.3 Les 4 types de métadonnées

> **DÉFINITION : Métadonnée**
> Une donnée qui **décrit** une autre donnée. Sans métadonnées, la colonne `CD_ST = 3` est incompréhensible. Avec : "Statut client, 3 = Prospect, propriétaire Direction Marketing".

Type	Question	Qui la remplit ?	Exemple
Technique	Comment est-elle structurée ?	Automatique (ingestion)	`monthly_income`, INTEGER, NOT NULL
Métier	Que signifie-t-elle ?	Les experts métier (vous)	"Salaire mensuel brut de l'employé"
Opérationnelle	Comment est-elle utilisée ?	Automatique (pipelines)	Source CSV Kaggle, 1 470 lignes, chargé le 28/10
Gouvernance	Qui décide, quelles contraintes ?	Owner / Steward	Owner Direction RH, Confidential, PII Oui, High

3.4 Catalogue, dictionnaire, classification

Notion	Niveau	Exemple
Data catalogue	Le dataset entier	"Ce dataset contient les états financiers SEC, owner Direction Finance, Public"
Dictionnaire de données	Une colonne	"`cik` : identifiant unique de l'entreprise à la SEC, 10 chiffres, NOT NULL"
Classification	Confidentialité	Public, Internal, Confidential, Restricted
Sensibilité	Criticité	Low, Medium, High
PII	Donnée personnelle ?	Oui si elle permet d'identifier une personne (âge + genre + département...)

3.5 Master Data et MDM

> **DÉFINITION : Master Data (donnée de référence maître)**
> Une entité **partagée** par plusieurs applications, **stable** dans le temps et **critique**. Exemple : un client, une entreprise, un produit. À l'inverse, une vente ou un cours de bourse du jour sont des données **transactionnelles** (des événements).

> **DÉFINITION : MDM et Golden Record**
> Le **MDM** (Master Data Management) rassemble les versions d'une même entité venant de plusieurs sources, les nettoie, les dédoublonne et publie **une seule version fiable** : le **Golden Record**. Toutes les applications pointent ensuite vers lui (par un identifiant, ex. `company_id`).

Les 5 tests du cours pour savoir si une entité mérite le MDM :

Test	Question	Exemple OUI	Exemple NON
Nature	Entité (qui / quoi) ou événement (quand) ?	Une entreprise	Un cours de bourse du 12 mars
Volatilité	La valeur change-t-elle rarement ?	Le nom d'une entreprise	Le prix d'une action
Réutilisation	Référencée par beaucoup d'enregistrements ?	Une entreprise a des milliers de faits financiers	
Partage	Plusieurs applications en ont besoin ?	SEC et S&P 500 parlent des mêmes entreprises	
Criticité	Une erreur coûte cher ?	Un mauvais rapprochement fausse une analyse	La couleur d'un bouton

Règle de gouvernance du cours : un domaine maître = un référentiel = un Data Owner.

Domaines de Master Data (chapitre 2) : **Parties** (personnes physiques ou morales), **Things** (produits, actifs), **Locations** (lieux), **Finance** (comptes), **Reference Data** (codes : devises, pays, secteurs).

---

4. Étapes

ÉTAPE 0 : Prérequis

Sur chaque machine :

- Docker Desktop lancé, avec **au moins 6 Go de RAM** (Settings, Resources)
- Git et un compte GitHub ajouté comme collaborateur du dépôt
- Python 3.11 ou plus
- VS Code (conseillé)

```
docker --version
git --version
python --version
```

> **CAPTURE D'ÉCRAN REQUISE**
> Le terminal montrant les trois versions.

ÉTAPE 1 : Récupérer le dépôt et créer sa branche

```
cd $HOME
git clone https://github.com/gatsoundoujuniior-netizen/multi-agent-ai-governance.git
cd multi-agent-ai-governance
git checkout -b finance        # ou investment, ou hr
```

> **EXPLICATION : pourquoi une branche par membre ?**
> `main` contient seulement du travail **relu et validé**. Chacun travaille sur sa branche, puis ouvre une **Pull Request** (PR). Un autre membre la relit avant la fusion dans `main`. C'est la "relecture croisée" exigée par le sujet.

ÉTAPE 2 : Lancer le PostgreSQL du projet

```
copy .env.example .env
docker compose up -d --wait
docker exec gouv_postgres psql -U gouv -d governance -c "\dn"
```

Vous devez voir les 4 schémas `data`, `gov`, `master`, `ref`.

> **EXPLICATION : les 4 schémas**
> Un schéma PostgreSQL est un "dossier" de tables dans la base. `ref` = données de référence (devises, pays, codes). `master` = les Golden Records du MDM. `data` = les données métier (brutes et nettoyées). `gov` = les tables de gouvernance (partie 2). Le port est **5433** (et non 5432) pour ne pas entrer en conflit avec une autre base sur votre PC.

ÉTAPE 3 : Lancer OpenMetadata (rappel du TP1)

```
docker compose -f openmetadata/docker-compose.yml up --detach --wait
docker compose -f openmetadata/docker-compose.yml ps
```

Ouvrir http://localhost:8585, se connecter avec `admin@open-metadata.org` / `admin`, puis **changer le mot de passe**.

> **ATTENTION**
> Le premier démarrage prend 5 à 15 minutes. Un conteneur qui redémarre au début n'est pas forcément une erreur. Si `openmetadata_elasticsearch` s'arrête, augmentez la RAM de Docker. Chaque membre a **son propre OpenMetadata local**. Pour que nos trois catalogues soient identiques, l'étape 5 se fait **de la même façon** sur les trois machines.

ÉTAPE 4 : Fixer les standards communs

Le sujet demande que l'on définisse **ensemble** des standards. Relisez ce tableau à trois, modifiez si besoin, puis cochez.

✓	Standard	Valeur proposée
☐	Schémas	`ref`, `master`, `data`, `gov`
☐	Nommage	minuscules, `snake_case`, clé primaire `<entite>_id` (ex. `company_id`)
☐	Tables brutes	`data.<dataset>_raw` (copie fidèle du fichier, jamais modifiée)
☐	Dates	format `YYYY-MM-DD`
☐	Classification	`Public`, `Internal`, `Confidential`, `Restricted`
☐	Sensibilité	`Low`, `Medium`, `High`
☐	Identifiants d'agent	`finance_agent`, `investment_agent`, `hr_agent`
☐	Noms des datasets	`sec_financial_statements`, `sp500_market_data`, `ibm_hr_analytics`
☐	Purposes valides (partie 2)	`valuation`, `reporting`, `audit`
☐	Seuil qualité	95 % de complétude minimum, 1 % de doublons maximum
☐	Fichiers SQL	`sql/1x_finance_*.sql`, `sql/2x_investment_*.sql`, `sql/3x_hr_*.sql`
☐	Scripts Python	`scripts/load_<dataset>.py`
☐	Dictionnaires	`docs/dictionnaire/<dataset>.md`
☐	Référentiels partagés	`ref.currency`, `ref.country` : Finance. `ref.sector` (GICS) : Investment

ÉTAPE 5 : Configurer OpenMetadata de la même façon (chacun sur sa machine)

5.1 Classifications (menu **Gouverner, Classification**, bouton **Ajouter**) :

Classification	Tags à créer
`Confidentialite`	`Public`, `Internal`, `Confidential`, `Restricted`
`Sensibilite`	`Low`, `Medium`, `High`

La classification **PII** existe déjà dans OpenMetadata (`PII.Sensitive` , `PII.NonSensitive`) : on la réutilise.

5.2 Équipes (menu **Paramètres, Équipes et utilisateurs**) : créer 3 équipes de type *Department* : `Direction Finance` , `Direction Investissement` , `Direction RH` . Ce seront les **Data Owners**.

5.3 Utilisateurs : créer un utilisateur par membre et lui donner le rôle **Data Steward**.

5.4 Domaines (menu **Marketplace de données, Domaines**) : créer `Finance` , `Investissement` , `RH` .

5.5 Glossaire (menu **Gouverner, Glossaire**) : créer un glossaire nommé `Glossaire Projet` . Chaque membre y ajoutera ses termes dans son TP.

5.6 Service PostgreSQL (menu **Paramètres, Services, Bases de données**, **Ajouter**) :

Champ	Valeur
Type	Postgres
Nom du service	`gouv_postgres`
Username / Password	`gouv` / le mot de passe de votre `.env`
Host and Port	`host.docker.internal:5433`
Database	`governance`

> **EXPLICATION : pourquoi `host.docker.internal` ?**
> OpenMetadata tourne **dans** un conteneur. Pour lui, `localhost` désigne le conteneur lui-même, pas votre PC. `host.docker.internal` est l'adresse que Docker Desktop donne à votre PC depuis l'intérieur des conteneurs.

Cliquez sur **Tester la connexion** : le test doit être vert. On lancera l'ingestion dans chaque TP, quand il y aura des tables.

> **CAPTURE D'ÉCRAN REQUISE**
> La page Classification avec `Confidentialite` et `Sensibilite` , et le test de connexion vert du service.

ÉTAPE 6 : Règles de travail en équipe

1. **Une branche par membre** : `finance` , `investment` , `hr` .
2. **Commits petits et clairs**, en français : `TP1 etape 2 : chargement brut du dataset SEC` .
3. **Pull Request à la fin de chaque étape**, relue par un autre membre :

Auteur	Relecteur
Junior (Finance)	Ivan (Investment)
Ivan (Investment)	Mohammed (HR)
Mohammed (HR)	Junior (Finance)

4. **Jamais de données dans Git** : `data/raw/` est ignoré. Chacun télécharge son dataset.
5. **Jamais le fichier `.env` dans Git**.
6. **Un seul fichier commun modifié à la fois** (README, standards) : prévenez les autres sur le groupe.

> **EXPLICATION : que vérifie le relecteur ?**
> Qu'il **comprend** ce qui a été fait et qu'il pourrait l'expliquer à l'oral. S'il ne comprend pas, l'auteur doit améliorer les commentaires ou la documentation. Le sujet l'exige : à la fin, chacun doit pouvoir expliquer tout le système.

---

5. Questions de compréhension (à préparer pour l'oral)

1. Pourquoi un agent IA ne doit-il jamais lire les données directement ?
2. Quelle est la différence entre un Data Owner et un Data Steward ? Pourquoi l'Owner n'est-il jamais l'IT ?
3. Donnez un exemple de métadonnée de chacun des 4 types pour **votre** dataset.
4. Un cours de bourse du jour est-il une Master Data ? Justifiez avec les 5 tests.
5. Pourquoi les trois membres doivent-ils utiliser exactement les mêmes noms de classification ?

6. Livrables du TP0

- [ ] Captures d'écran des étapes 0, 2, 3 et 5
- [ ] Tableau des standards coché et validé par les 3 membres
- [ ] Chaque membre a sa branche poussée sur GitHub

Suite : chaque membre passe à son TP (TP1 Finance, TP2 Investment, TP3 HR).